

052-Linux中的文件系统

{% hint style="info" %} 尽量让 GPT 根据目前的状况重写了下面的内容，PPT {% endhint %}

磁盘结构

- #0 引导块
- #1 超级块：存放文件系统管理信息
 - inode表所占盘块数
 - 文件数据所占盘块数
 - 主存中登记的空闲盘块数
 - 主存中登记的空闲块物理块号
 - 主存中登记的空闲inode数
 - 主存中登记的空闲inode编号
- #2-k+1 索引块
- #k+2-n 数据块

GPT说：

在现代 Linux（如 ext2/3/4）中，磁盘被划分为多个块组（block group），每组都包含：

- 超级块副本
- 块位图
- inode 位图
- inode 表
- 数据块区

只有第一个块组的第一个超级块（块号1）是主超级块。

inode & dentry

- inode: inode号 | 文件类型 | 权限 | 拥有者 | 大小 | 数据块地址指针 | 创建/修改时间 | inode引用数 | 所在设备 | ...
- inode 存在 inode 表中
- 目录块中存放 inode 编号，根据编号找到对应 inode
- i_data[]: inode的索引
 - 前 12 项：直接索引
 - 第 13 项：一级间接索引
 - 第 14 项：二级间接索引
 - 第 15 项：三级间接索引
- dentry 目录项: 文件名 | inode号

用户打开文件表

- 用户&进程级，记录当前进程的文件，位于PCB中
- 表项: 文件描述符，指向系统打开文件表内的对应表项
- 文件描述符为int类型，实现按号存取

系统打开文件表

- 内核级，记录系统中所有已打开的文件
- 表项：文件指针 | 偏移量 | `f_count` | 访问模式 | `inode`表指针

活动 inode 表

- 内存区开辟，硬盘目录项的 Cache
- 若访问文件时，`inode`不在内存中，在活动表中开辟一个 `inode` 结构，读入磁盘中的 `inode`
- 关闭文件时：减少 `inode` 的引用计数 `i_count`
- 多用户同时使用文件：对当前使用用户数进行引用计数，若为 0 则写回
- 路径解析、`ls`、`stat` 会加载 `inode`，此时用户打开文件表无信息，只有活动 `inode` 表有信息

总体关系

- 文件描述符 `fd` -> 用户打开文件表项 -> 系统打开文件表项 -> 活动 `inode` 表项 -> 磁盘上的 `inode` 表项 -> 物理块

文件共享

- 硬链接：指向目标文件的 `inode`
 - `ln file1 file2`：创建硬链接
 - 在 `inode` 中维护文件引用计数，当引用计数为 0 时，删除文件，否则只删除目录项
- 软链接：仅仅指向文件名的路径，而非`inode`号
 - `ln -s file1 file2`：创建软链接
 - 软链接的 `inode` 和源文件的 `inode` 不同
 - 跨文件系统只能使用软链接

类型	是否独立文件	是否共享 <code>inode</code>	是否跨文件系统	是否跟随原文件
硬链接	否	是	否	是
软链接/符号链接/快捷方式	是	否（有自己 <code>inode</code> ）	是	否（指向路径）

文件动态共享

- 共享文件：多个用户/进程同时使用同一文件

情况	文件偏移量是否共享	说明
<code>fork()</code>	☑ 共享偏移量	父子进程共享系统打开文件表项
<code>dup()</code> / <code>dup2()</code>	☑ 共享偏移量	两个文件描述符指向同一系统打开文件表项
再次 <code>open()</code>	✗ 不共享偏移量	每次 <code>open</code> 新建系统打开文件表项
不同用户进程	✗ 不共享偏移量	各自独立打开文件

总结：常见的引用计数

引用计数类型	所在结构	意义	增加时机	减少时机
--------	------	----	------	------

引用计数类型	所在结构	意义	增加时机	减少时机
硬链接计数 (link count)	inode (磁盘)	文件名 (目录项) 指向该 inode 的数量	创建硬链接 (<code>ln file1 file2</code>)	删除硬链接 (或文件)
打开文件计数 (open count)	活动 inode 表 (内存)	有多少打开的文件表项指向这个 inode	打开文件 (<code>open()</code>)	关闭文件 (<code>close()</code>)
文件表项引用计数 (file descriptor count) <code>f_count</code>	系统打开文件表	有多少用户文件描述符指向这个表项	<code>dup()</code> , <code>fork()</code> 复制父进程文件描述符	<code>close()</code> 或进程结束
目录项引用计数	VFS 层/目录缓存	有多少目录项 (dentry) 指向某个 inode (含子目录)	访问目录/路径解析	文件删除/目录项被淘汰
inode 引用计数 (内存引用计数) <code>i_count</code>	活动 inode 表	有多少结构在使用这个 inode (dentry、open file 等)	文件被访问 (打开、路径遍历)	文件句柄关闭、路径释放
挂载点引用计数 (mount count)	内核挂载表	文件系统或挂载点被引用的次数	被挂载、访问文件系统	卸载或不再访问

{% hint style="warning" %} 区分 `i_count` 和 `f_count`

- `i_count`: inode 级别
- `f_count`: 文件级别
- 文件和 inode 并非一一映射, 所以 `f_count` 不一定和 `i_count` 一致 {% endhint %}

主存映射文件

- 降低访问磁盘的时间开销
- 新增系统调用:
 - `mmap()`: 将文件映射到内存中
 - `munmap()`: 解除映射

虚拟文件系统

- 屏蔽不同文件系统, 本地/远程文件系统的差异, 统一接口
- 层次
 - 应用层: 系统调用, `open()`、`read()`、`write()`、`close()`
 - 系统接口层: `sys_open()`、`sys_read()`、`sys_write()`、`sys_close()`
 - 实现层: 具体的文件系统实现
- VFS 层
 - 超级块 (整个文件系统): `struct super_block`
 - 文件对象: `struct file`
 - 索引节点: `struct inode`
 - 目录项对象: `struct dentry`
- 注册/注销文件系统类型: 编译时确定/`insmod`注册注销

- 文件系统安装: `mount`, 将文件系统挂载到某个目录下
- 文件系统卸载: `umount`, 将文件系统从某个目录下卸载